LEXICON

GIT & GITHUB

Git and GitHub

Version control, and working together with feature branches

LEXICON

Goals

- After this session you can:
  - Explain what version control is
  - Tell Git and GitHub apart
  - Describe the everyday Git commands
  - Work with feature branches and pull requests
  - Solve a merge conflict

LEXICON

Recap

- Agile = small steps, fast feedback, welcome change
- The Manifesto: 4 values, 12 principles
- Scrum: Product Owner, Scrum Master, Developers
- Events: Sprint, Planning, Daily Scrum, Review, Retrospective
- Artifacts: Product Backlog, Sprint Backlog, Increment
- Kanban: a board, WIP limits, continuous flow

LEXICON

Quick questions

1. What are the three roles (accountabilities) in a Scrum Team?
2. What is the difference between the Sprint Review and the Retrospective?
3. What is a WIP limit, and why is it useful?

LEXICON

Why version control?

report.docx, report_v2.docx, report_final.docx, report_FINAL_really.docx

- See the full history: what changed, when, by whom and why
- Go back to a version that worked
- Many people work on the same code at the same time
- A backup on another computer

LEXICON

Git and GitHub

Git	GitHub
A program on your computer	A website that stores Git repositories
Works without internet	Lets teams share code online
Saves the history of your code	Adds pull requests, reviews, issues, boards
Made by Linus Torvalds, 2005	Owned by Microsoft. Similar: GitLab, Bitbucket

LEXICON

Words you need

Word	Meaning
Repository (repo)	A project folder that Git keeps track of
Commit	A saved snapshot of the project, with a message
Branch	A separate line of work
Remote	A copy of the repo somewhere else, like GitHub
Clone	Download a repo to your computer
Push / Pull	Send commits to / get commits from the remote
Merge	Combine the work of two branches
Pull request (PR)	A request to merge your branch, with review

LEXICON

Three places for your code

```
Working directory  --git add-->  Staging area  --git commit-->  Repository
(your files)                     (ready to save)                  (history)
```

- Working directory: the files you edit
- Staging area: the changes you choose for the next commit
- Repository: all your saved commits

LEXICON

Getting started

```
# Once per computer: who are you?
git config --global user.name "Anna Lindqvist"
git config --global user.email "anna@example.com"
git config --global init.defaultBranch main

# Start a new repo in a folder ...
git init

# ... or download one from GitHub
git clone https://github.com/anna/webshop.git
```

LEXICON

Everyday commands

```
git status                          # what has changed?
git add main.py                     # stage one file
git add .                           # stage all changes
git commit -m "Add product search"
git log --oneline                   # show the history
git diff                            # show exactly what changed
```

- git status is your best friend: run it all the time

LEXICON

Good commits

Bad	Good
fixed stuff	Fix wrong total when cart is empty
update	Add category filter to product list
asdfgh	Remove unused import in shop.py
One commit with a whole week of work	Many small commits, one change each

- Write the message like an order: 'Add ...', 'Fix ...', 'Remove ...'

LEXICON

Files Git should ignore

```
# .gitignore
__pycache__/
.venv/
.env
*.log
```

- A file called .gitignore lists files Git should never track
- Never commit passwords or API keys
- Once it's pushed, it's in the history, even if you delete it later

LEXICON

Your computer and GitHub

```
git remote -v                # where is the remote?
git push -u origin main      # first push of a branch
git push                     # send my commits
git pull                     # get new commits and merge them
git fetch                    # only download, don't merge
```

- origin = the usual nickname for the GitHub repo
- Commits stay on your computer until you push

LEXICON

Signing in to GitHub

- Your GitHub password does not work in the terminal (since 2021)
- Windows: Git Credential Manager opens a browser login
- Or: a personal access token, used instead of the password
- Or: an SSH key (common for developers)
- Two-factor authentication is required on GitHub

LEXICON

What is a branch?

```
main:       A---B---C-----------F
                     \         /
feature:              D----E---

D and E are made on the feature branch.
F = the merge: the feature is now in main.
```

- A branch = a separate line of work
- main should always work
- New work happens on its own branch, then gets merged

LEXICON

Branch commands

```
git branch                          # list branches (* = where you are)
git switch -c feature/search        # create a branch and go to it
git switch main                     # go back to main
git merge feature/search            # merge it into the branch you're on
git branch -d feature/search        # delete it when it's merged
```

- Older guides use git checkout -b instead of git switch -c. Both work.

LEXICON

The feature branch workflow

1. Get the latest main: git switch main, git pull
2. Create a branch: git switch -c feature/search
3. Work and commit, in small steps
4. Push the branch: git push -u origin feature/search
5. Open a pull request on GitHub
6. A teammate reviews it
7. Merge it into main
8. Delete the branch and pull main again

LEXICON

Naming branches

Type	Example
New feature	feature/product-search
Bug fix	bugfix/cart-total
Small fix	fix/typo-in-readme
With an issue number	feature/12-order-history

- One feature per branch
- Short-lived: days, not months
- Lowercase, with dashes, no spaces

LEXICON

Pull requests

- 'Please review my branch and merge it into main'
- A title and a description: what and why
- Shows every changed line
- Teammates comment and approve
- Automatic checks can run, like tests
- 'Closes #12' closes issue 12 when it's merged

LEXICON

Code review

As a reviewer	As the author
Be kind and specific	Keep the PR small
Ask questions: 'Why ... ?'	Explain what and why in the description
Check: does it work? Is it readable?	Don't take comments personally
Approve when it meets the Definition of Done	Answer every comment

LEXICON

Merging a pull request

Button on GitHub	What happens
Create a merge commit	All commits are kept, plus one merge commit
Squash and merge	All commits become one single commit in main
Rebase and merge	The commits are put on top of main, no merge commit

- Your team decides which one to use

LEXICON

Merge conflicts

```
<<<<<<< main
print("Welcome to the shop!")
=======
print("Welcome to our webshop!")
>>>>>>> feature/new-welcome
```

- Two branches changed the same lines differently
- Git can't guess which one is right, so it asks you
- Fix: choose, remove the markers, then git add and git commit

LEXICON

Fewer conflicts

- Pull main often, and bring it into your branch
- Keep branches small and short-lived
- Talk to the team: 'I'm changing the login page today'
- Don't reformat whole files you're not working on
- Commit and push often

LEXICON

Protecting main

- On GitHub, you can add rules to the main branch:
  - No pushing directly to main: always a pull request
  - At least one approval before merging
  - All checks (like tests) must pass
- Then main always works

LEXICON

Git and Scrum together

Scrum	GitHub
User story	Issue
Sprint Backlog board	GitHub Projects board
Working on a story	A feature branch
'Code reviewed' in the DoD	An approved pull request
Increment	What's merged into main

LEXICON

Oops! Undoing things

```
git restore main.py             # throw away my changes in a file
git restore --staged main.py    # un-stage it (keep the changes)
git commit --amend -m "..."      # fix the last commit (before push!)
git revert a3f9c21              # undo a commit with a new commit
```

- After pushing: use revert, never rewrite shared history
- Be careful with git reset --hard and git push --force

LEXICON

Remember

- Git saves the history of your code. GitHub shares it with your team
- Everyday: git status, add, commit, push, pull
- Commit small, with clear messages. Never commit secrets
- New work = a new branch. main always works
- Feature branch, then pull request, review and merge
- A merge conflict is a question, not an error